

Dataproc: Cluster vs Serverless Batches

Arquitetura Pub/Sub → Cloud Run → Dataproc

Comparação de Características

Critério	Dataproc Cluster	Serverless Batches
INFRAESTRUTURA		
Provisionamento	Manual — define master, workers, tipo de máquina. Tempo ~2–5 min.	Automático. Nenhum cluster para gerenciar. Inicia em ~30–60 s.
Ciclo de vida	Persistente ou efêmero. Requer gestão ativa.	Totalmente gerenciado. Infra destruída ao fim do batch.
Rede / VPC	Suporte completo a Shared VPC, subrede customizada, Private Google Access.	Suporta Shared VPC. Requer subnetwork explícita.
Customização SO	Alta — init actions, imagens custom, pacotes via pip/apt.	Limitada — via custom container (Docker) ou dependências no config.
CUSTO		
Modelo de cobrança	Por VM ativa (vCPU + memória + disco). Cobra mesmo ocioso.	Por DCU (Dataproc Compute Unit) consumida. Paga só o que roda.
Workloads esporádicos	Ineficiente — cluster parado gera custo. Mitigado com clusters efêmeros.	Ideal — zero custo quando não há batches rodando.
Workloads contínuos	Mais eficiente — custo previsível, overhead amortizado.	Pode ser mais caro — overhead por DCU em jobs muito frequentes.
PERFORMANCE		
Autoscaling	Via Autoscaling Policies (YARN metrics). Requer tuning.	Automático e transparente. GCP escala sem configuração.
Latência de startup	2–5 min para cluster frio. Persistentes têm latência zero.	30–90 s por batch. Sem opção de manter quente.
Jobs interativos	Suportado — Jupyter/Workbench, SSH, Spark shell.	Não suportado — apenas jobs batch assíncronos.
OPERAÇÃO		
Terraform	google_dataproc_cluster — recurso estável e maduro.	google_dataproc_batch — disponível, menos parâmetros expostos.
Cloud Composer	CreateCluster + SubmitJob + DeleteCluster operators (3 tasks).	DataprocCreateBatchOperator — 1 task por job.
IAM / SA	SA do cluster precisa de roles em Compute, Storage etc.	SA de runtime do batch. Requer roles/dataproc.worker + acesso aos recursos alvo.
Observabilidade	Spark History Server nativo. YARN UI disponível.	Spark History Server gerenciado automaticamente pelo GCP.

Recomendação Geral

Para pipelines orientados a eventos (Pub/Sub → Cloud Run → Dataproc), o **Serverless Batches** é a escolha natural: cada evento dispara um batch isolado, sem cluster pré-aquecido, e o custo é zero entre execuções. Reserve clusters para desenvolvimento interativo (Jupyter/Workbench) ou workloads contínuos onde o overhead de startup seria amortizado.

Padrão recomendado: desenvolvimento e exploração em cluster efêmero via Terraform, produção via `DataprocCreateBatchOperator` no Composer 3 ou diretamente pelo Cloud Run.

Arquitetura: Pub/Sub → Cloud Run → Dataproc Serverless

O fluxo de eventos passa por três camadas principais, cada uma com responsabilidades distintas:

Pub/Sub	Entrega at-least-once. O <code>message_id</code> do evento deve ser usado como <code>batch_id</code> determinístico para garantir idempotência.
Cloud Run	Consome o evento, valida, e chama a API <code>dataproc.projects.locations.batches.create</code> . Roda com SA próprio (<code>roles/dataproc.editor</code>).
Serverless Batch	Job Spark isolado por evento. SA de runtime separado com <code>roles/dataproc.worker</code> + acesso a GCS, BigQuery etc.
Shared VPC	O batch config deve especificar <code>subnetwork</code> explicitamente. Private Google Access habilitado na subrede.

Decisões de Design

Fire-and-forget vs. Polling	O Cloud Run pode submeter o batch e retornar 200 imediatamente, ou fazer polling do status. Se o job demorar mais que 60 min (timeout máx do Run), use Workflows ou outro tópico Pub/Sub para polling externo.
Idempotência	Use <code>batch_id</code> determinístico baseado no <code>message_id</code> do Pub/Sub. O Dataproc rejeita batches com ID duplicado nativamente, resolvendo o problema de at-least-once delivery.
SA Chain	Cloud Run SA → <code>roles/dataproc.editor</code> para submeter o batch. Batch runtime SA → <code>roles/dataproc.worker</code> + acesso a GCS, BigQuery, Secret Manager etc.
Shared VPC	Especificar <code>subnetwork</code> explicitamente no batch config. Serverless não usa rede default. Private Google Access obrigatório na subrede.

Exemplo: Cloud Run submetendo batch (Python)

Exemplo de código Python para o Cloud Run submeter um Dataproc Serverless Batch com `batch_id` idempotente baseado no `message_id` do Pub/Sub:

```
from google.cloud import dataproc_v1

def submit_batch(event_data: dict, message_id: str):
    client = dataproc_v1.BatchControllerClient()
    batch_id = f"batch-{message_id}" # idempotente
    batch = dataproc_v1.Batch()
    batch.pyspark_batch = dataproc_v1.PySparkBatch(
```

```
main_python_file_uri="gs://bucket/job.py",
)
batch.runtime_config = dataproc_v1.RuntimeConfig(
version="2.2",
)
batch.environment_config = dataproc_v1.EnvironmentConfig(
execution_config=dataproc_v1.ExecutionConfig(
service_account="batch-sa@project.iam.gserviceaccount.com",
subnetwork_uri="projects/host/regions/us-east1/subnetworks/subnet",
)
)
request = dataproc_v1.CreateBatchRequest(
parent="projects/PROJECT/locations/REGION",
batch=batch,
batch_id=batch_id,
)
operation = client.create_batch(request=request)
return operation # fire-and-forget ou aguardar com .result()
```